

Since your goal is **Codeforces + DSA in C++**, I would optimise for **contest readiness**, not “becoming a C++ developer”.

The target is:

In 4 weeks, be comfortable solving Div 2 A/B/C problems without STL slowing you down.

Phase 1: STL Foundations (Week 1)

Goal: Become fluent with the STL containers and algorithms used in 80% of CP problems.

Day 1 — Vectors, Pairs, Iterators

Learn

- vector
- pair
- iterators
- range-based loops
- auto

Resources

Video

[Luv STL Playlist - Vector and Pair Videos](#)

Reading

[USACO Guide STL Basics](#)

Practice

Solve 5–10 problems requiring:

```
vector<int>  
pair<int,int>  
sort()
```

Day 2 — Sets and Maps

Learn

```
set  
multiset  
map  
unordered_map
```

Important Functions

```
insert  
erase  
find  
count
```

Resources

Video

Continue Luv playlist.

Reading

[cppreference Containers](#)

Practice

Frequency counting problems.

Typical pattern:

```
map<int,int> freq;  
freq[x]++;
```

Day 3 — Algorithms

Learn

sort
reverse
find
count
binary_search
lower_bound
upper_bound

Resource

[cppreference Algorithms Library](#)

Practice

Search and sorting problems.

Day 4 — Stack, Queue, Deque

Learn

stack
queue
deque

Practice

Simple simulation problems.

Day 5 — Priority Queue

Learn

priority_queue<int>

and

```
priority_queue<
    int,
    vector<int>,
    greater<int>
>
```

Practice

Top K elements.

Day 6–7 — STL Revision

Build a cheatsheet.

For every STL container write:

Declaration
Insertion
Deletion
Search
Complexities

Phase 2: Competitive Programming STL (Week 2)

Now start learning the STL pieces that appear constantly in contests.

Day 8

Learn

string
stringstream

Resource:

[cppreference String Library](#)

Day 9

Learn

`next_permutation`

Practice

Permutation generation problems.

Day 10

Learn

Custom comparators.

```
sort(v.begin(), v.end(),
    [](int a, int b){
        return a>b;
    });
```

Day 11

Learn

`vector<pair<int,int>>`

Sorting pairs.

Day 12

Learn

`multiset`

Very common in Div2 C/D.

Day 13

Learn

`unordered_map`
`unordered_set`

Understand average $O(1)$.

Day 14

Revision.

Attempt a 2-hour virtual contest.

Use only STL.

Phase 3: DSA + STL Together (Week 3)

Now STL becomes a tool instead of the topic.

Binary Search

Resource:

[USACO Guide Binary Search](#)

Learn:

lower_bound
upper_bound

inside real problems.

Prefix Sums

Resource:

[CP Handbook \(free PDF\)](#).

Read:

- Prefix sums
 - Range queries
-

Two Pointers

Resource:

[USACO Guide Two Pointers](#)

Sliding Window

Often uses:

map
unordered_map
deque

Phase 4: Contest Readiness (Week 4)

Now stop learning STL.

Start solving.

Daily

2 Codeforces A Problems

2 Codeforces B Problems

1 CSES Problem

Resources

CSES Problem Set

[CSES Problem Set](#)

This is probably the best free DSA practice source.

Codeforces

[Codeforces Problemset](#)

CP Handbook

[Competitive Programmer's Handbook PDF](#)

Read alongside solving.

Resources Ranked

If I had to pick only 5:

1. Luv STL Playlist

[Luv STL Playlist](#)

2. USACO Guide

[USACO Guide](#)

3. CSES Problem Set

[CSES Problem Set](#)

4. Competitive Programmer's Handbook

[Competitive Programmer's Handbook](#)

5. cppreference

[cppreference](#)

STL Mastery Checklist

Before moving to harder DSA, you should be able to write from memory:

vector
pair
map
set
multiset
unordered_map
unordered_set
stack
queue
deque
priority_queue

sort
binary_search
lower_bound
upper_bound
next_permutation

lambda comparator

auto
range based loops

Once these are second nature, the highest ROI topics are:

1. Binary Search
2. Prefix Sums
3. Two Pointers
4. Sliding Window
5. Greedy
6. Recursion & Backtracking
7. Trees
8. Graphs
9. DP

That's roughly the progression followed by many strong Codeforces users. You can paste this directly into a document and export it as a PDF study plan.